

MANUAL DE UNITY Y GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para el desarrollo colaborativo de videojuegos en Microsoft Windows con PowerShell 7+, Git Credential Manager, UnityYAMLMerge y GameCI

[Git LFS & Assets Binarios](#)[UnityYAMLMerge 3-Way Driver](#)[Visible Meta Files & GUIDs](#)[Multi-Scene & Prefab Isolation](#)[GameCI & GitHub Actions](#)[Catálogo de Incidentes Críticos](#)

Manual de Unity y GitHub: De Novato a Avanzado en PowerShell (Windows)

Plataforma: Microsoft Windows 10 / Windows 11 / Windows Server

Motor: Unity 2022 LTS / Unity 6 (6000.x)

Herramientas: PowerShell 7+, Git for Windows 2.40+, Git LFS, GitHub CLI (`gh`), GitHub Desktop, Git Credential Manager (GCM), Unity Hub, UnityYAMLMerge

Índice de Contenidos

1. **Parte I: Fundamentos y Preparación del Entorno Unity en Windows**
 - 1.1 Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora
 - 1.2 Instalación de Git, Git LFS, GitHub CLI y GitHub Desktop con Winget
 - 1.3 Configuración Crítica del Editor: Visible Meta Files y Force Text
 - 1.4 La Regla de Oro de los Archivos `.meta` y los GUIDs
 - 1.5 El Archivo `.gitignore` Oficial y Optimizado para Unity en Windows
 - 1.6 Configuración Exhaustiva de Git LFS con `.gitattributes`
 - 1.7 Arquitectura de Proyectos AAA: Separación de Assets Propios (`_Project/`) vs Plugins
2. **Parte II: Flujo de Trabajo Esencial Diario (Nivel Novato)**
 - 2.1 Inicializar y Publicar un Proyecto en GitHub (PowerShell y GitHub Desktop)
 - 2.2 Clonación Correcta de Proyectos con Git LFS en Windows
 - 2.3 El Ciclo de Trabajo Seguro: Modificar, Inspeccionar y Confirmar Commits
 - 2.4 Conventional Commits Aplicados al Desarrollo de Videojuegos
 - 2.5 Sincronización sin Romper la Cache (`pull --rebase` vs Fetch en Desktop)
3. **Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)**
 - 3.1 Estrategia de Ramas en Equipos de Videojuegos (PowerShell y Desktop)
 - 3.2 Arquitectura de Escenas Divididas (Multi-Scene Editing Aditivo)
 - 3.3 Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes
 - 3.4 Configuración de UnityYAMLMerge en Windows y Personalización de Fallback en `mergespecfile.txt`
 - 3.5 Uso de Git Stash, Historial y Reversión Segura (PowerShell y Desktop)
 - 3.6 Modularización de Código con Assembly Definitions (`.asmdef` y `.asmref`)
 - 3.7 Unity Accelerator: Aceleración de Descargas y Caché de Importación en LAN
4. **Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente**

- 4.1 Tema 1: Prevención Arquitectónica de Conflictos en Unity
 - 4.2 Tema 2: Conflictos en Archivos `.meta` (GUID Desincronizado)
 - 4.3 Tema 3: Conflictos en Scripts C# (`.cs`)
 - 4.4 Tema 4: Conflictos en Escenas y Prefabs con UnityYAMLMerge (PowerShell y Desktop)
 - 4.5 Tema 5: Forzar una Versión Completa de Asset (`--ours` vs `--theirs`)
 - 4.6 Tema 6: Cambios Locales en el Editor al Hacer Pull
 - 4.7 Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS
 - 4.8 Tema 8: Conflictos en Binarios y Bloqueo con Git LFS Lock
 - 4.9 Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano
5. Parte V: Herramientas Modernas de Productividad Avanzada
- 5.1 Git Worktrees en Windows: Trabajar en Múltiples Ramas sin Recargar `Library/`
 - 5.2 Depuración Binaria con Git Bisect y Blame en C#
 - 5.3 GitHub Codespaces desde PowerShell
 - 5.4 GitHub Copilot CLI en PowerShell
 - 5.5 Git Hooks y Validación Pre-commit de Archivos `.meta` en Windows
 - 5.6 Descargas Parciales y Ahorro de Cuota de Git LFS con `lfs.fetchexclude` y `git sparse-checkout`
6. Parte VI: Gestión de Paquetes UPM y Dependencias
- 6.1 Instalación de Paquetes Mediante URLs de Git en UPM
 - 6.2 Creación y Publicación de Paquetes UPM en Repositorios Privados con Tokens
 - 6.3 Addressables Asset System frente a `Resources/` : Versionado y Despliegue en CDN
7. Parte VII: Automatización CI/CD con GitHub Actions y GameCI
- 7.1 Arquitectura de GameCI para StandaloneWindows64
 - 7.2 Activación de Licencias de Unity en GitHub Actions
 - 7.3 Pipeline Automatizado: Tests y Compilación StandaloneWindows64 (.exe)
 - 7.4 Subida Automática de Instaladores a GitHub Releases
 - 7.5 Pruebas Automatizadas con Code Coverage y Reportes en GitHub Actions
8. Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos
- 8.1 Branch Protection Rules y Rulesets en Windows
 - 8.2 Gestión de Secretos para APIs de Juegos (Steam, Photon, Azure)
 - 8.3 Gobernanza con `CODEOWNERS` para Artistas y Programadores
9. Parte IX: Distribución y Despliegue con GitHub Releases
- 9.1 Creación de Releases con Tags Semánticos desde PowerShell
 - 9.2 Empaquetado y Publicación de Archivos ZIP e Instaladores de Windows

10. Parte X: Catálogo Maestro de Incidentes Críticos de Unity en Windows

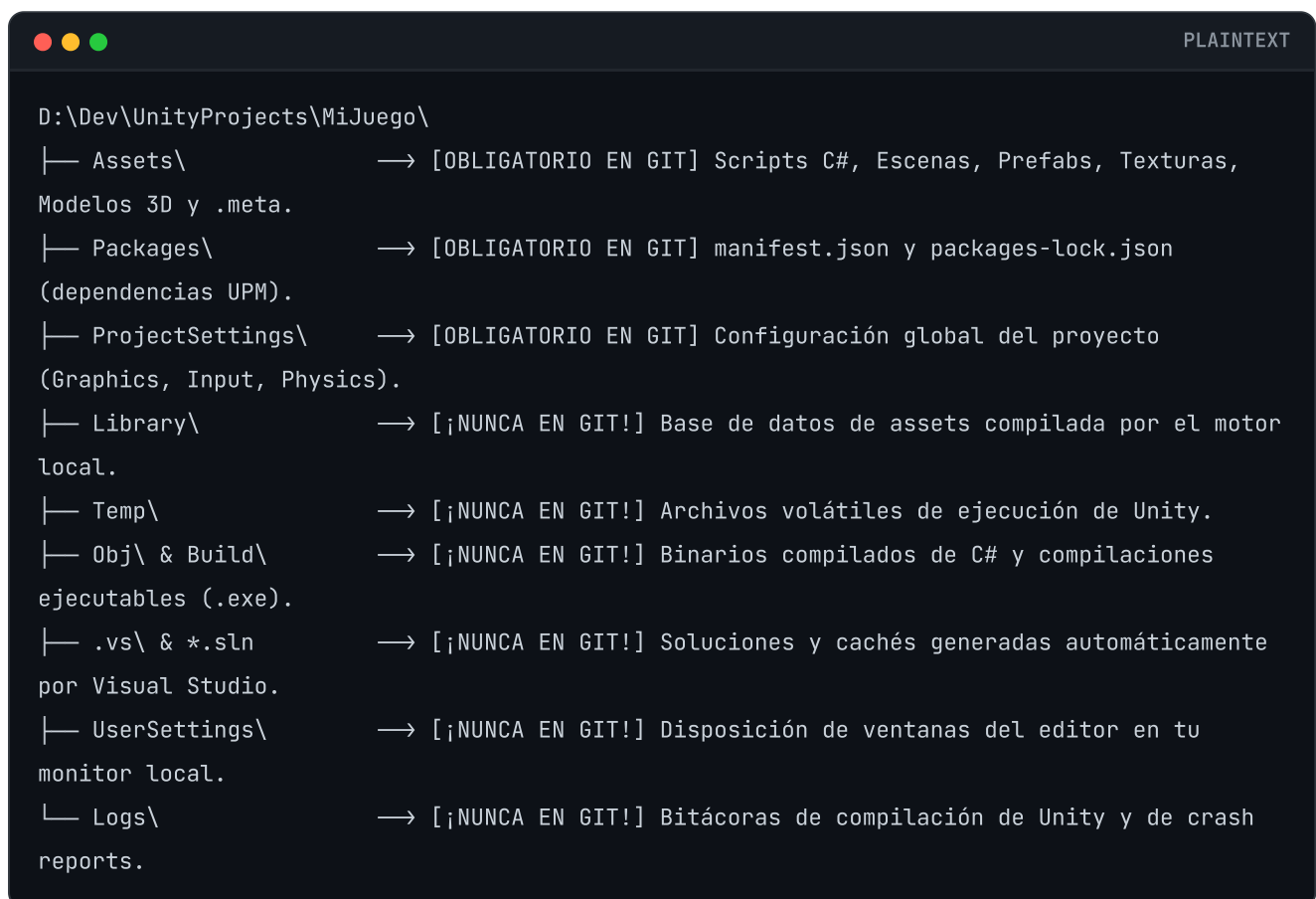
- 10.1 Incidente 1: Bloqueo de Archivos por el Proceso de Unity (`unLink failed` / `Permission Denied`)
 - 10.2 Incidente 2: Longitud de Ruta Máxima de Windows Excedida (`Filename too long`)
 - 10.3 Incidente 3: Corrupción de Finales de Línea CRLF vs LF en Scripts C# y YAML
 - 10.4 Incidente 4: "Missing Script" Masivo por Desincronización de GUIDs
 - 10.5 Incidente 5: Subida Accidental de `Library/` o Archivos Temporales de Visual Studio
 - 10.6 Incidente 6: Push Rechazado por Archivo > 100 MB Atrapado en Commits Locales
 - 10.7 Incidente 7: Límite de Ancho de Banda de Git LFS Superado
 - 10.8 Incidente 8: Conflictos de Visual Studio con `.csproj` y `.sln` en Git
 - 10.9 Incidente 9: Fuga de Claves de Steamworks o Azure en ScriptableObjects
 - 10.10 Incidente 10: Regeneración Segura y Limpia de la Caché Local en Windows
-

Parte I: Fundamentos y Preparación del Entorno

Unity en Windows

1.1 Anatomía de un Proyecto de Unity: Qué se versiona y qué se ignora

En Windows, los proyectos de Unity generan decenas de miles de archivos temporales al compilar con Visual Studio o Rider:



```
D:\Dev\UnityProjects\MiJuego\  
├─ Assets\           → [OBLIGATORIO EN GIT] Scripts C#, Escenas, Prefabs, Texturas,  
Modelos 3D y .meta.  
├─ Packages\         → [OBLIGATORIO EN GIT] manifest.json y packages-lock.json  
(dependencias UPM).  
├─ ProjectSettings\  
(Graphics, Input, Physics).  
├─ Library\          → [¡NUNCA EN GIT!] Base de datos de assets compilada por el motor  
local.  
├─ Temp\             → [¡NUNCA EN GIT!] Archivos volátiles de ejecución de Unity.  
├─ Obj\ & Build\  
ejecutables (.exe).  
├─ .vs\ & *.sln      → [¡NUNCA EN GIT!] Soluciones y cachés generadas automáticamente  
por Visual Studio.  
├─ UserSettings\  
monitor local.  
└─ Logs\             → [¡NUNCA EN GIT!] Bitácoras de compilación de Unity y de crash  
reports.
```

[↑ Volver al Índice de Contenidos](#)

1.2 Instalación de Git, Git LFS, GitHub CLI y GitHub Desktop con Winget

Abre **PowerShell 7** (o Windows Terminal) como Administrador y ejecuta:

```
# 1. Instalar Git for Windows, Git LFS, GitHub CLI y GitHub Desktop
winget install --id Git.Git -e --source winget
winget install --id GitHub.cli -e --source winget
winget install --id GitHub.GitHubDesktop -e --source winget

# 2. Inicializar Git LFS globalmente
git lfs install

# 3. Configurar soporte para rutas largas en Windows (>260 caracteres)
git config --system core.longpaths true

# 4. Habilitar Git Credential Manager para Windows
git config --global credential.helper manager
```

¿Qué hace este comando?

Instala el motor completo de Git, GitHub CLI, GitHub Desktop con soporte para el administrador de credenciales de Windows (GCM), activa los filtros globales de Git LFS y desbloquea el límite histórico de 260 caracteres de Windows NTFS (`core.longpaths true`), vital para proyectos complejos de Unity con carpetas profundamente anidadas.

[↑ Volver al Índice de Contenidos](#)

1.3 Configuración Crítica del Editor: Visible Meta Files y Force Text

Antes de agregar el proyecto a Git, verifica estos parámetros en Unity:

1. Abre tu proyecto en Unity en Windows.
2. Navega al menú superior: **Edit** → **Project Settings** → **Version Control**.
 - En el campo **Mode**, selecciona: `Visible Meta Files`.
3. Navega a **Edit** → **Project Settings** → **Editor**.
 - En **Asset Serialization Mode**, selecciona: `Force Text`.

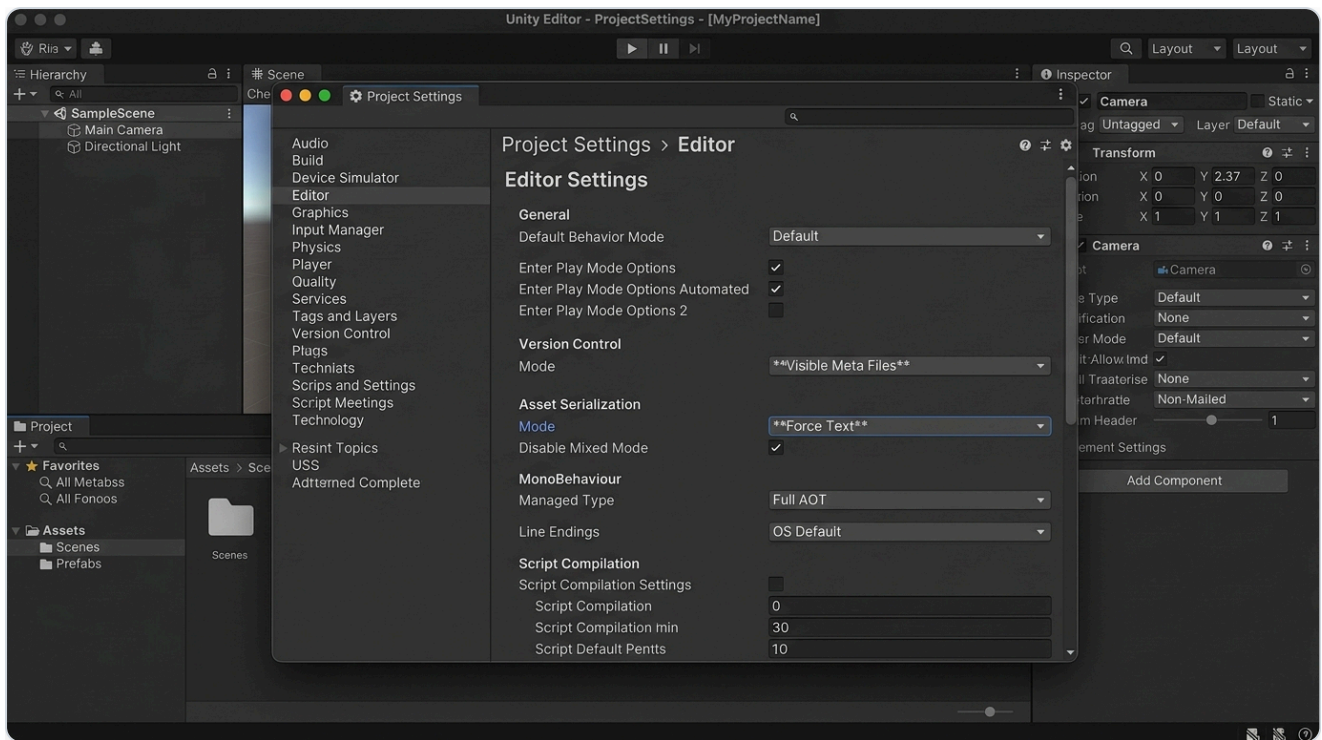


Figura 1.1: Configuración obligatoria en Unity Project Settings (Editor) estableciendo Visible Meta Files y Force Text para serialización YAML en Windows.

IMPORTANT

Force Text obliga al motor a serializar todas las escenas (`.unity`), prefabs (`.prefab`) y materiales (`.mat`) en texto plano **YAML legible** en lugar del formato binario propietario de Unity, permitiendo a Git generar diffs precisos y resolver conflictos.

[↑ Volver al Índice de Contenidos](#)

1.4 La Regla de Oro de los Archivos `.meta` y los GUIDs

Cada asset en `Assets\` cuenta con un archivo gemelo `.meta` que contiene su GUID único:

```
fileFormatVersion: 2
guid: 3f8a92b104c84a56a1b2c3d4e5f60718
```

CAUTION

LA REGLA FUNDAMENTAL:

Si mueves, renombras o eliminas un asset desde el Explorador de Windows o la consola de PowerShell, **debes mover, renombrar o eliminar idénticamente su archivo `.meta`**. Nunca hagas un commit de un asset sin su respectivo `.meta`.

[↑ Volver al Índice de Contenidos](#)

1.5 El Archivo `.gitignore` Oficial y Optimizado para Unity en Windows

Crea el archivo `.gitignore` en la raíz del proyecto:

```
# =====  
# .gitignore para Unity en Microsoft Windows  
# =====  
  
[Ll]ibrary/  
[Tt]emp/  
[Oo]bj/  
[Bb]uild/  
[Bb]uilds/  
[Ll]ogs/  
[Uu]ser[Ss]ettings/  
[Mm]emoryCaptures/  
  
# Asset server cache  
sysinfo.txt  
*.stackdump  
  
# Visual Studio / VS Code / JetBrains Rider  
.vs/  
.idea/  
*.csproj  
*.unityproj  
*.sln  
*.suo  
*.user  
*.userprefs  
*.pidb  
*.booproj  
*.svd  
*.pdb  
*.opendb  
*.VC.db  
  
# Compilaciones Windows (.exe / .dll)  
*.exe  
*.dll  
*.apk  
*.unitypackage  
  
# Archivos del Explorador de Windows  
Thumbs.db
```

```
ehthumbs.db  
[Dd]esktop.ini  
$RECYCLE.BIN/
```

[↑ Volver al Índice de Contenidos](#)

1.6 Configuración Exhaustiva de Git LFS con `.gitattributes`

Guarda este archivo `.gitattributes` en la raíz del repositorio:

```
# =====
# Configuración Git LFS para Unity en Windows
# =====

# Formatos 3D y Rigs
*.fbx filter=lfs diff=lfs merge=lfs -text
*.obj filter=lfs diff=lfs merge=lfs -text
*.blend filter=lfs diff=lfs merge=lfs -text
*.dae filter=lfs diff=lfs merge=lfs -text
*.max filter=lfs diff=lfs merge=lfs -text

# Texturas e Imágenes Pesadas
*.psd filter=lfs diff=lfs merge=lfs -text
*.tga filter=lfs diff=lfs merge=lfs -text
*.png filter=lfs diff=lfs merge=lfs -text
*.jpg filter=lfs diff=lfs merge=lfs -text
*.jpeg filter=lfs diff=lfs merge=lfs -text
*.exr filter=lfs diff=lfs merge=lfs -text
*.hdr filter=lfs diff=lfs merge=lfs -text
*.tif filter=lfs diff=lfs merge=lfs -text
*.tiff filter=lfs diff=lfs merge=lfs -text

# Audio y Vídeo
*.wav filter=lfs diff=lfs merge=lfs -text
*.mp3 filter=lfs diff=lfs merge=lfs -text
*.ogg filter=lfs diff=lfs merge=lfs -text
*.mp4 filter=lfs diff=lfs merge=lfs -text
*.mov filter=lfs diff=lfs merge=lfs -text

# Archivos Comprimidos
*.zip filter=lfs diff=lfs merge=lfs -text
*.7z filter=lfs diff=lfs merge=lfs -text

# Bloqueo Concurrente para Binarios
*.fbx lockable
*.psd lockable
*.blend lockable
*.wav lockable

# Normalización de Saltos de Línea y Merge Driver de Unity
*.cs text diff=csharp eol=lf
```

```
*.json text eol=lf
*.shader text eol=lf
*.unity merge=unityyamlmerge eol=lf
*.prefab merge=unityyamlmerge eol=lf
*.mat merge=unityyamlmerge eol=lf
*.asset merge=unityyamlmerge eol=lf
*.meta text eol=lf
```

[↑ Volver al Índice de Contenidos](#)

1.7 Arquitectura de Proyectos AAA: Separación de Assets Propios (`\_Project/`) vs Plugins

En estudios profesionales de desarrollo de videojuegos sobre Windows, colocar assets directamente en la raíz de `Assets\` genera colisiones permanentes. Cuando importas paquetes de la Unity Asset Store, sus estructuras se mezclan con el código de tu juego, oscureciendo los diffs de Git y complicando la auditoría de ramas.

Patrón Recomendado: Estructura Raíz Prefijada

Organiza tu proyecto en Windows creando una carpeta contenedora principal con un guion bajo para forzar su aparición al inicio de la jerarquía de Unity:

```
Assets\
├─ _Project\           → [CÓDIGO, ESCENAS Y ASSETS PROPIOS DEL ESTUDIO]
│   ├─ Art\            → Modelos 3D, Texturas PBR, Materiales y Rigs
│   ├─ Audio\          → Pistas musicales y efectos WAV
│   ├─ Core\           → GameManagers, Servicios, Singletons y Arquitectura
│   ├─ Gameplay\       → Controladores, Mecánicas, Inventario y Físicas
│   ├─ Prefabs\        → Variantes y Prefabs modulares
│   ├─ Scenes\         → Escenas maestras y aditivas
│   └─ UI\             → Menús Canvas, TextMeshPro Assets y Sprites
├─ Plugins\           → Librerías nativas (.dll de C++ o wrappers de Windows)
└─ ThirdParty\        → Herramientas y paquetes adquiridos en Asset Store (solo
lectura)
```

 TIP

Aislar el código en `Assets\_Project\` permite que los archivos `.gitignore` y las políticas de `CODEOWNERS` se mantengan concisas. Asimismo, puedes actualizar o eliminar paquetes externos en `ThirdParty\` sin tocar accidentalmente tus scripts ni desencadenar resoluciones complejas de GUIDs.

[↑ Volver al Índice de Contenidos](#)

Parte II: Flujo de Trabajo Esencial Diario (Nivel Novato)

2.1 Inicializar y Publicar un Proyecto en GitHub (PowerShell y GitHub Desktop)

Modalidad A: Vía PowerShell 7



```
Set-Location "D:\Dev\UnityProjects\MiJuego"
git init
git lfs install
git add .gitattributes .gitignore
git commit -m "chore: inicializar configuracion Unity con LFS y gitignore"
git branch -M main
gh repo create MiJuegoWindows --private --source=. --remote=origin --push
```

¿Qué hace este comando?

Navega a la carpeta del proyecto, inicializa Git y Git LFS, confirma los archivos de configuración base y usa `gh` para crear el repositorio privado en tu cuenta de GitHub y subir la rama `main`.

Modalidad B: Vía GitHub Desktop

1. Abre **GitHub Desktop**.
2. Pulsa en **File** → **Add Local Repository** (o `Ctrl + O`).
3. Haz clic en **Choose...** y selecciona la carpeta de tu proyecto Unity.
4. Si la carpeta aún no tiene Git inicializado, pulsa en el enlace azul **"create a repository"**.
5. Haz clic en el botón azul **Add Repository**.
6. En la barra superior, haz clic en **Publish repository** para subirlo a tu cuenta de GitHub de forma privada o pública.

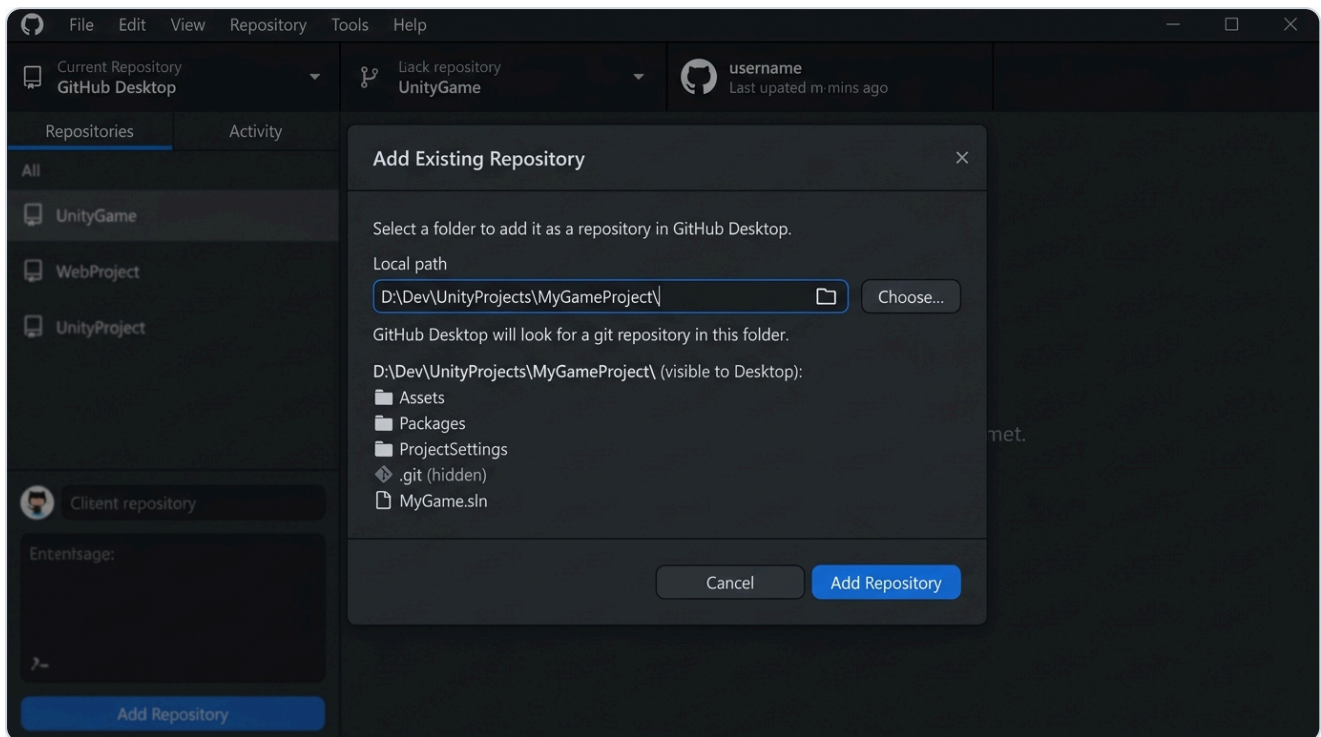


Figura 2.1: Cuadro de diálogo 'Add Existing Repository' en GitHub Desktop vinculando la carpeta del proyecto Unity en Windows.

[↑ Volver al Índice de Contenidos](#)

2.2 Clonación Correcta de Proyectos con Git LFS en Windows

Modalidad A: Vía PowerShell

```
POWERSHELL

git lfs install
git clone git@github.com:mi-organizacion/videojuego.git
Set-Location videojuego
git lfs pull
```

Modalidad B: Vía GitHub Desktop

1. Ve a **File** → **Clone Repository** (o `Ctrl + Shift + 0`).
2. Selecciona el repositorio de la lista de tu cuenta o introduce la URL.
3. Elige la carpeta de destino local y pulsa **Clone**.
4. GitHub Desktop descargará automáticamente todos los modelos 3D y texturas de LFS en segundo plano.

2.3 El Ciclo de Trabajo Seguro: Modificar, Inspeccionar y Confirmar Commits

Modalidad A: Vía PowerShell



```
POWERSHELL

# 1. Comprobar archivos modificados
git status

# 2. Agregar scripts y prefabs junto con sus respectivos .meta
git add Assets/Scripts/PlayerController.cs Assets/Scripts/PlayerController.cs.meta
git add Assets/Prefabs/Player.prefab Assets/Prefabs/Player.prefab.meta

# 3. Confirmar commit
git commit -m "feat(player): agregar control de movimiento y fisica con Rigidbody2D"
```

Modalidad B: Vía GitHub Desktop

1. En la pestaña **Changes** de la izquierda, revisa la lista de archivos modificados.
2. Comprueba que cada script `.cs` o prefab tenga marcada su casilla junto a su archivo gemelo `.meta`.
3. Haz clic sobre cualquier archivo para ver en el panel derecho las diferencias exactas de código (*diff*).
4. Rellena el campo **Summary** con el mensaje del commit y opcionalmente añade detalles en **Description**.
5. Pulsa en el botón azul **Commit to main** (o a la rama de trabajo activa).

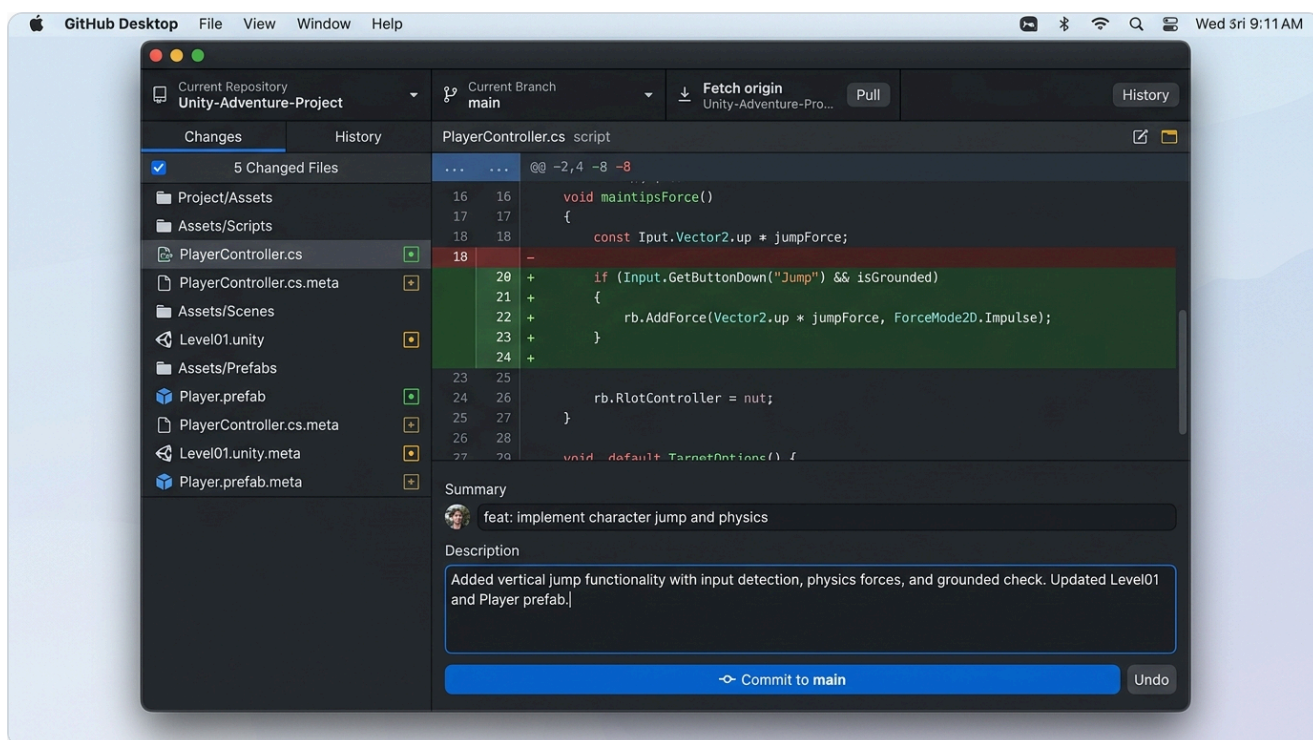


Figura 2.2: Pestaña de cambios en GitHub Desktop verificando la integridad en pareja de archivos y metadatos de Unity antes de confirmar.

[↑ Volver al Índice de Contenidos](#)

2.4 Conventional Commits Aplicados al Desarrollo de Videojuegos

Prefijo	Área de Aplicación en Videojuegos	Ejemplo
feat:	Nuevas mecánicas o sistemas jugables	feat(inventory): crear sistema de slots y arrastre de items
fix:	Corrección de fallos o glitches de físicas	fix(camera): corregir vibracion de Cinemachine al seguir al jugador
art:	Modelos 3D, texturas, animaciones y rigs	art(weapons): anadir modelo 3D y textura PBR de la escopeta
level:	Modificaciones de escenas y diseño de niveles	level(arena): distribuir spawners de oleadas y luces nocturnas
audio:	Efectos de sonido y mezclas de AudioMixer	audio(ambient): integrar pistas de viento y pasos sobre nieve
perf:	Optimizaciones de draw calls y memoria	perf(occlusion): hornear matrices de Occlusion Culling en nivel 1
chore:	Mantenimiento de paquetes UPM o gitignore	chore(upm): actualizar paquete Input System a version 1.7.0

[↑ Volver al Índice de Contenidos](#)

2.5 Sincronización sin Romper la Cache (`pull --rebase` vs Fetch en Desktop)

Modalidad A: Vía PowerShell

```
git pull --rebase origin main
git push origin main
```

El uso de `--rebase` evita los merge commits artificiales que alteran la fecha de modificación de los archivos y obligan al motor a reimportar la base de datos de assets.

Modalidad B: Vía GitHub Desktop

1. Pulsa en **Fetch origin** en la barra superior para buscar actualizaciones remotas.

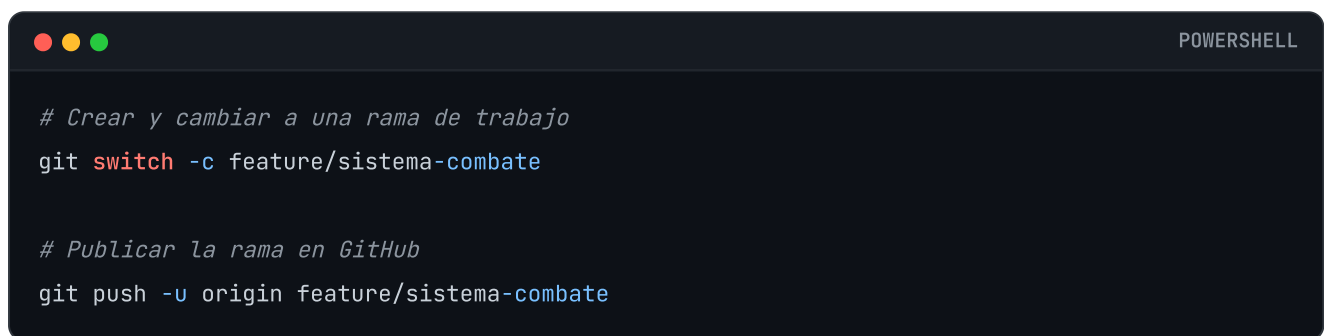
2. Si hay commits nuevos de tus compañeros, pulsa en el botón **Pull origin**.
3. Envía tus cambios pendientes haciendo clic en **Push origin**.

[↑ Volver al Índice de Contenidos](#)

Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)

3.1 Estrategia de Ramas en Equipos de Videojuegos (PowerShell y Desktop)

Modalidad A: Vía PowerShell



```
# Crear y cambiar a una rama de trabajo
git switch -c feature/sistema-combate

# Publicar la rama en GitHub
git push -u origin feature/sistema-combate
```

Modalidad B: Vía GitHub Desktop

1. En la barra superior, haz clic en el menú **Current Branch**.
2. Escribe el nombre de tu rama (ej. `feature/sistema-combate`).
3. Haz clic en el botón azul **New branch....**
4. Haz clic en **Publish branch** para sincronizarla con GitHub.

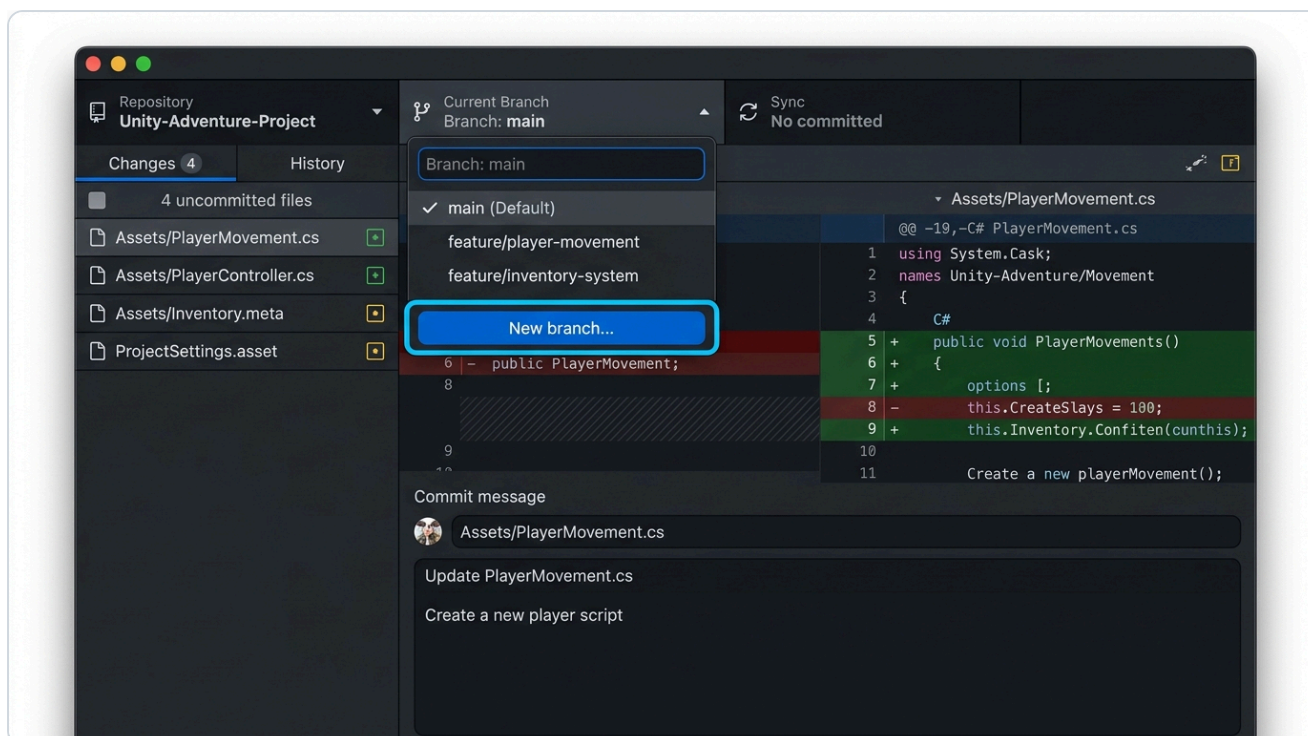


Figura 3.1: Menú desplegable 'Current Branch' en GitHub Desktop facilitando la creación y conmutación de ramas aisladas en Windows.

[↑ Volver al Índice de Contenidos](#)

3.2 Arquitectura de Escenas Divididas (Multi-Scene Editing Aditivo)

En videojuegos, editar la misma escena al mismo tiempo genera colisiones catastróficas. La solución estándar en la industria es dividir el mundo en múltiples escenas aditivas:

```

Nivel_Bosque\
├─ Nivel_Core.unity      → Cámaras, GameManagers, UI Canvas.
├─ Nivel_Arte.unity     → Terreno, vegetación, rocas y props (Artistas).
├─ Nivel_Luces.unity    → Post-processing, luces y sondas (Iluminadores).
└─ Nivel_Gameplay.unity → Triggers, enemigos y zonas de muerte (Diseñadores).
  
```

Script C# para Carga Aditiva en Tiempo de Ejecución:

Guarda en `Assets\Scripts\AdditiveLoader.cs` :

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class AdditiveLoader : MonoBehaviour
{
    [SerializeField] private string[] subScenes = { "Nivel_Arte", "Nivel_Luces",
    "Nivel_Gameplay" };

    private void Start()
    {
        foreach (string scene in subScenes)
        {
            if (!SceneManager.GetSceneByName(scene).isLoaded)
            {
                SceneManager.LoadSceneAsync(scene, LoadSceneMode.Additive);
            }
        }
    }
}
```

[↑ Volver al Índice de Contenidos](#)

3.3 Aislamiento de Trabajo Mediante Prefabs Anidados y Variantes

- Edita componentes siempre en el **Prefab Mode**, no directamente en la escena.
- Emplea **Prefab Variants** para variantes de enemigos o props. Las variantes solo guardan las diferencias (*overrides*), evitando conflictos de fusión en el archivo base.

[↑ Volver al Índice de Contenidos](#)

3.4 Configuración de UnityYAMLMerge en Windows y Personalización de Fallback en `mergespecfile.txt`

UnityYAMLMerge es el motor oficial de resolución tridireccional desarrollado por Unity para interpretar la jerarquía de nodos YAML de escenas y prefabs.

Paso 1: Configurar el Driver en Git con PowerShell

```
POWERSHELL

# Localizar dinámicamente la versión más reciente del editor en Program Files
$UnityEditorDir = Get-ChildItem "C:\Program
Files\Unity\Hub\Editor\*\Editor\Data\Tools\UnityYAMLMerge.exe" | Select-Object -Last 1

if ($UnityEditorDir) {
    $UnityPath = $UnityEditorDir.FullName
    git config --global merge.unityyamlmerge.name "Unity Smart Merge"
    git config --global merge.unityyamlmerge.driver "`"$UnityPath`" merge -p -- `"%0`"
`"%B`" `"%A`" `"%A`""
    git config --global merge.unityyamlmerge.trustExitCode true
    git config --global merge.unityyamlmerge.recursive binary
    Write-Host "✓ UnityYAMLMerge configurado exitosamente: $UnityPath" -ForegroundColor
Green
} else {
    Write-Warning "No se encontró Unity Editor en C:\Program Files\Unity\Hub\Editor\"
}
```

Paso 2: Personalización del Fallback Visual en `mergespecfile.txt`

En la misma carpeta donde reside `UnityYAMLMerge.exe` (`C:\Program Files\Unity\Hub\Editor\
<version>\Editor\Data\Tools\` ) se ubica el archivo `mergespecfile.txt` . Este archivo le indica a Unity
qué herramienta de fusión visual tridireccional debe abrir cuando dos personas editen simultáneamente
los mismos campos de un mismo GameObject:

```
# Configuración recomendada para Windows en mergespecfile.txt

# Opción A: Visual Studio 2022
* use "C:\Program Files\Microsoft Visual Studio\2022\Community\Common7\IDE\devenv.exe" /diff
"%b" "%t" "%d"

# Opción B: JetBrains Rider
* use "C:\Program Files\JetBrains\JetBrains Rider\bin\rider64.exe" merge "%b" "%t" "%d" "%d"

# Opción C: Beyond Compare 4
* use "C:\Program Files\Beyond Compare 4\BCompare.exe" "%b" "%t" "%d" "%d"

# Opción D: Visual Studio Code
* use "code" --wait --merge "%b" "%t" "%d" "%d"
```

NOTE

Al invocar `git mergetool -t unityyamlmerge`, Unity resolverá sin intervención humana el 95% de los conflictos limpios (por ejemplo, luces y cámaras añadidas concurrentemente). Si existe una colisión irresoluble en una misma propiedad, abrirá inmediatamente tu editor visual con los marcadores tridireccionales (*base*, *remote*, *local*).

[↑ Volver al Índice de Contenidos](#)

3.5 Uso de Git Stash, Historial y Reversión Segura (PowerShell y Desktop)

Modalidad A: Vía PowerShell



```
# Guardar cambios sin confirmar
git stash save "WIP: ajuste de inventario"

# Cambiar de rama y revisar bug
git switch main

# Restaurar cambios
git switch feature/mi-rama
git stash pop
```

Modalidad B: Vía GitHub Desktop

1. Al cambiar de rama con cambios sucios, pulsa en **"Leave my changes on [rama-actual]"** (crea un stash).
2. Para restaurar, pulsa en **Restore** dentro de **Stashed Changes** en la barra lateral.
3. Para deshacer un commit con errores: ve a la pestaña **History**, clic derecho sobre el commit → **Revert changes in commit**.

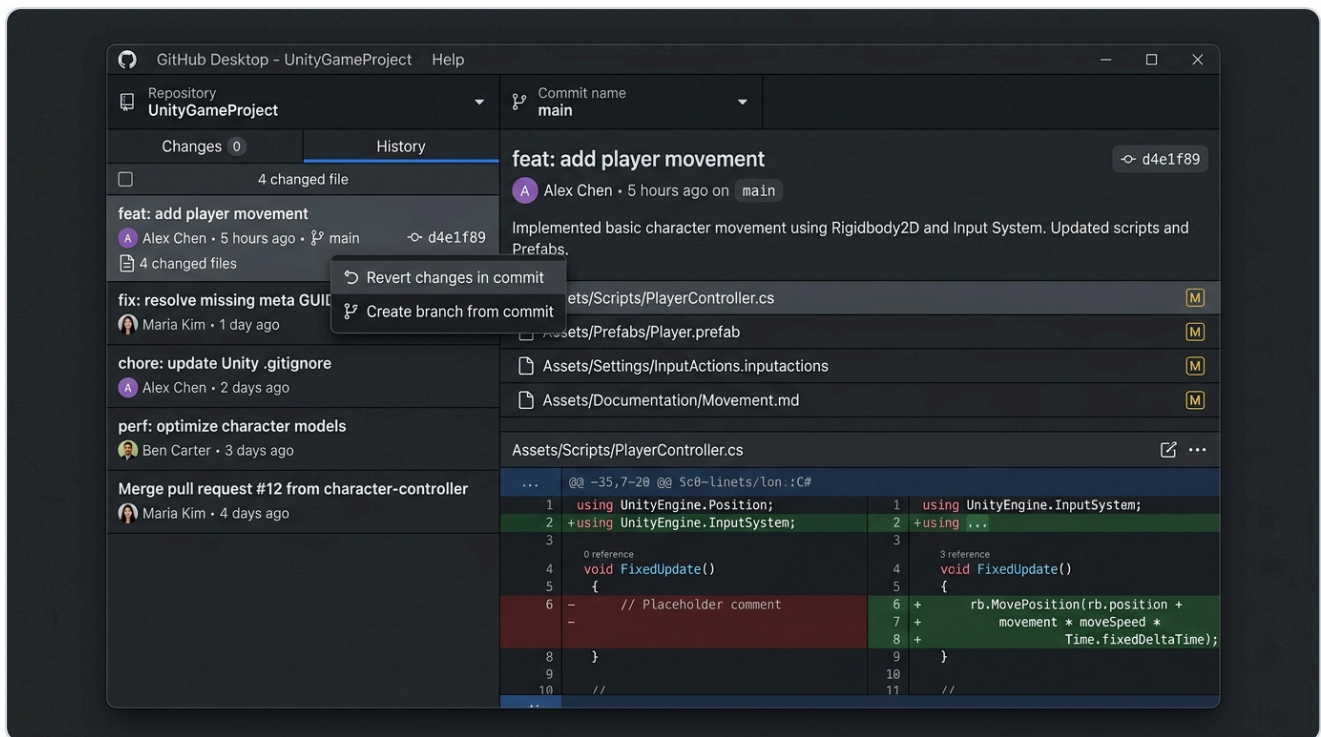


Figura 3.2: Pestaña 'History' en GitHub Desktop permitiendo inspección de diffs y reversión limpia de commits problemáticos.

[↑ Volver al Índice de Contenidos](#)

3.6 Modularización de Código con Assembly Definitions (`.asmdef` y `.asmref`)

En proyectos medianos y grandes de Windows, todos los scripts de C# se compilan por defecto en una librería masiva llamada `Assembly-CSharp.dll`. Cada vez que cualquier programador altera una línea o hace un pull, Unity compila todo el proyecto, bloqueando el Editor durante minutos.

La Solución Profesional: Archivos `.asmdef``

Desacopla tu código en ensamblados independientes creando archivos `.asmdef``:

- `Assets\_Project\Core\Game.Core.asmdef`
- `Assets\_Project\Gameplay\Game.Gameplay.asmdef` (con dependencia de `Game.Core`)
- `Assets\_Project\UI\Game.UI.asmdef` (con dependencia de `Game.Core`)
- `Assets\_Project\Editor\Game.Editor.asmdef` (con plataforma asignada únicamente a Editor)

```
{
  "name": "Game.Gameplay",
  "rootNamespace": "Game.Gameplay",
  "references": [
    "Game.Core",
    "Unity.InputSystem"
  ],
  "includePlatforms": [],
  "excludePlatforms": [],
  "allowUnsafeCode": false,
  "overrideReferences": false,
  "precompiledReferences": [],
  "autoReferenced": true,
  "defineConstraints": [],
  "versionDefines": [],
  "noEngineReferences": false
}
```

Beneficios en el Trabajo Diario con Git:

1. **Compilación en Milisegundos:** Un commit que modifique `Game.Gameplay` solo recompila esa DLL en Windows, sin congelar el IDE.
2. **Pull Requests Comprensibles:** El equipo de desarrollo identifica con precisión qué arquitectura fue alterada en el diff.
3. **Cero Dependencias Circulares:** El compilador de Roslyn rechaza cualquier referencia cíclica entre ensamblados.

[↑ Volver al Índice de Contenidos](#)

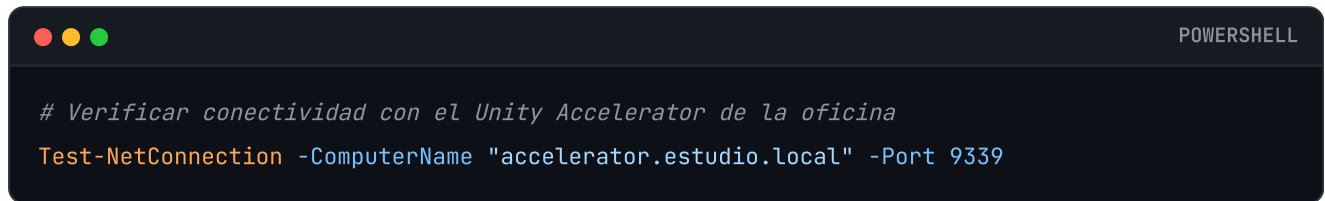
3.7 Unity Accelerator: Aceleración de Descargas y Caché de Importación en LAN

Cuando un equipo de desarrollo en Windows realiza `git pull` de modelos 3D y texturas 4K, cada estación de trabajo reimporta y comprime los assets en local mediante DirectX/Vulkan, saturando los núcleos de la CPU.

Unity Accelerator funciona como un servidor de caché proxy dentro de la red local del estudio:

1. El primer integrante del equipo que importa un asset nuevo sube los binarios transformados al Accelerator por el puerto 9339.

2. Al ejecutar `git pull` en las demás estaciones Windows, el Editor detecta que el hash del asset ya existe en el servidor local y lo descarga de inmediato a velocidad Gigabit/10Gbe, eliminando esperas de hasta 40 minutos.



```
# Verificar conectividad con el Unity Accelerator de la oficina
Test-NetConnection -ComputerName "accelerator.estudio.local" -Port 9339
```

[↑ Volver al Índice de Contenidos](#)

Parte IV: Soluciones por Temas a Conflictos y Edición Concurrente

4.1 Tema 1: Prevención Arquitectónica de Conflictos en Unity

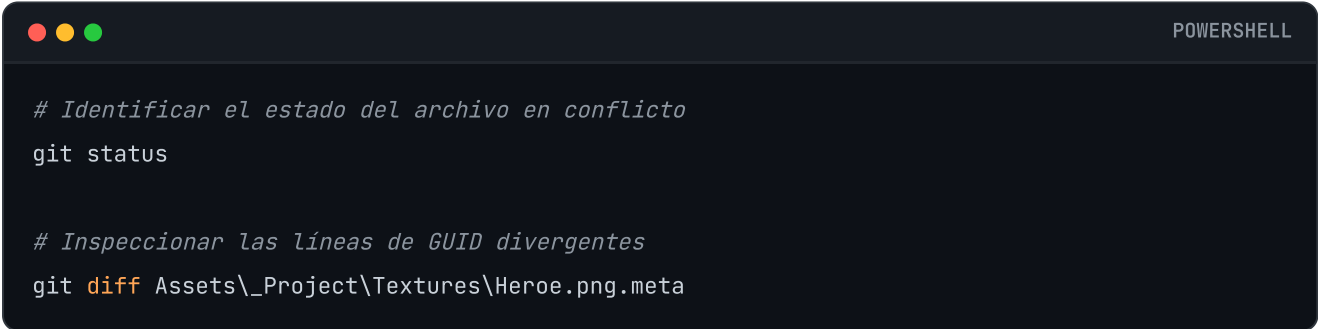
Los conflictos en Unity no deben resolverse con suerte; deben prevenirse por diseño:

1. **Regla de oro de escenas:** Jamás permitas que dos desarrolladores modifiquen la misma escena monolítica simultáneamente. Utiliza la arquitectura multi-escena aditiva explicada en la sección 3.2.
2. **Modularización en Prefabs:** Convierte cada subsistema visual y de juego en Prefabs anidados aislados.
3. **Coordinación de LFS Locks:** Comunica y bloquea activos binarios pesados antes de intervenirlos.

[↑ Volver al Índice de Contenidos](#)

4.2 Tema 2: Conflictos en Archivos `.meta` (GUID Desincronizado)

Se produce cuando dos miembros del equipo añaden un asset con el mismo nombre o mueven carpetas concurrentemente en Windows, generando dos GUIDs dispares para el mismo recurso:



```
# Identificar el estado del archivo en conflicto
git status

# Inspeccionar las líneas de GUID divergentes
git diff Assets\_Project\Textures\Heroe.png.meta
```

Solución Paso a Paso:

1. Si el recurso ya fue referenciado en escenas o prefabs por tu compañero en el servidor remoto, **conserva el GUID remoto**.
2. Abre el `.meta` en Visual Studio Code o Visual Studio y elimina los marcadores de conflicto dejando una sola línea `guid: .`
3. Confirma la resolución:

```
git add Assets\_Project\Textures\Heroe.png.meta
git commit -m "fix(meta): resolver colision de GUID en textura Heroe"
```

[↑ Volver al Índice de Contenidos](#)

4.3 Tema 3: Conflictos en Scripts C# (`.cs`)

Ocurre cuando dos programadores editan la misma sección o método dentro de un archivo C#:

```
<<<<<< HEAD
    void Jump() {
        rb.AddForce(Vector2.up * jumpForce, ForceMode2D.Impulse);
    }
=====
    void Jump() {
        if (isGrounded) {
            rb.linearVelocity = new Vector2(rb.linearVelocity.x, jumpSpeed);
        }
    }
>>>>>> feature/salto-mejorado
```

Solución:

1. Abre el script en Visual Studio 2022 o JetBrains Rider y unifica ambas intenciones:

```
void Jump() {
    if (isGrounded) {
        rb.AddForce(Vector2.up * jumpForce, ForceMode2D.Impulse);
    }
}
```

2. Guarda el archivo y confírmalo en PowerShell:

```
git add Assets\_Project\Scripts\PlayerController.cs  
git commit -m "fix(player): unificar validacion de suelo con impulso de salto"
```

[↑ Volver al Índice de Contenidos](#)

4.4 Tema 4: Conflictos en Escenas y Prefabs con UnityYAMLMerge (PowerShell y Desktop)

Cuando dos desarrolladores agregan o modifican GameObjects diferentes en la misma escena:

Modalidad A: Vía PowerShell con UnityYAMLMerge

```
git mergetool -t unityyamlmerge
```

¿Qué hace este comando?

Invoca `UnityYAMLMerge.exe`, el cual reconstruye el árbol AST de YAML y une los GameObjects agregados por ambas partes sin corromper los identificadores internos `fileID`.

Modalidad B: Vía GitHub Desktop

1. Tras un merge o rebase conflictivo, aparece el cuadro modal **Resolve conflicts before merging**.
2. En la lista de archivos con advertencia, pulsa en el menú desplegable junto a la escena `.unity`.
3. Selecciona **Open in UnityYAMLMerge** (o tu herramienta configurada en `mergespecfile.txt`).
4. Al completarse la fusión tridireccional, haz clic en el botón azul **Resolve Conflicts**.

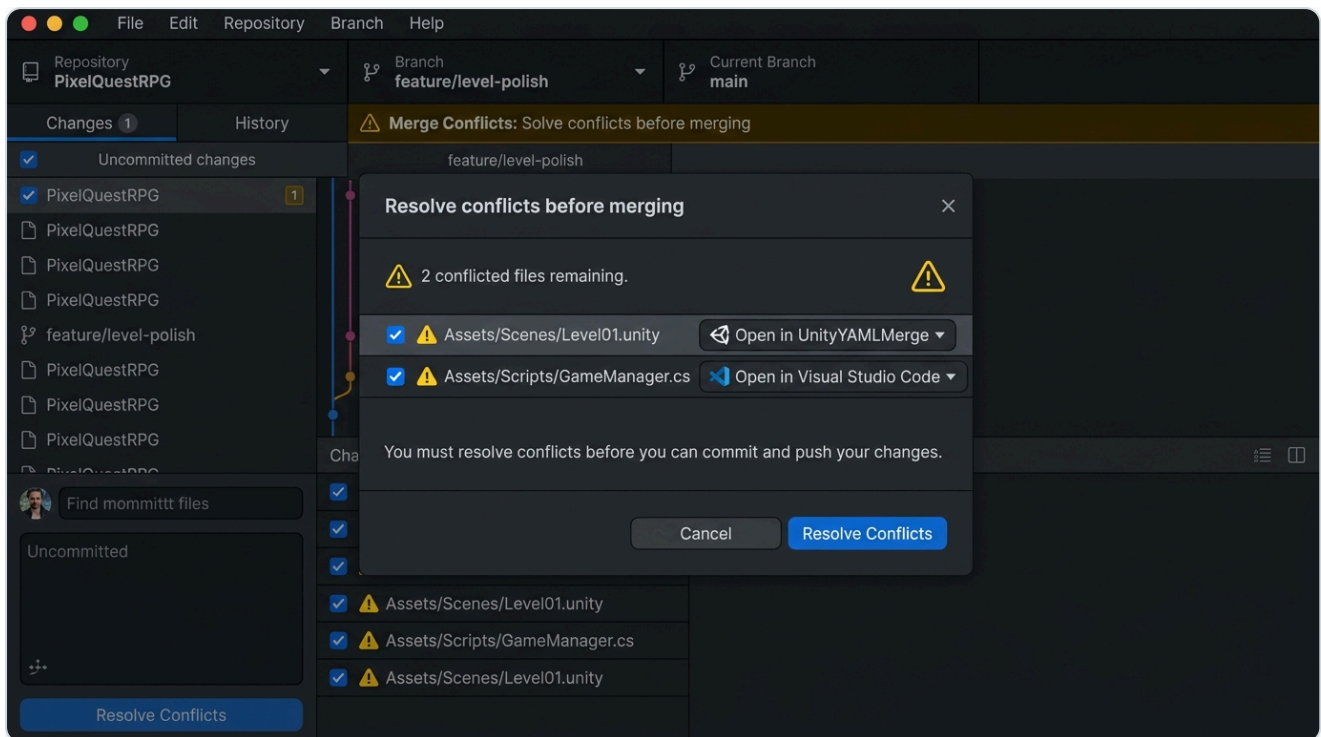


Figura 4.1: Modal de resolución de conflictos en GitHub Desktop en Windows con opciones de apertura en UnityYAMLMerge.

[↑ Volver al Índice de Contenidos](#)

4.5 Tema 5: Forzar una Versión Completa de Asset (`--ours` vs `--theirs`)

Si una escena o prefab se corrompió irreversiblemente durante una fusión y el equipo decide descartar por completo una de las dos versiones:

Modalidad A: Vía PowerShell

```
POWERSHELL

# Opción 1: Conservar tu versión local intacta y descartar la del servidor
git checkout --ours Assets\Project\Scenes\Nivel01.unity
git add Assets\Project\Scenes\Nivel01.unity
git commit -m "resolve: conservar version local de Nivel01"

# Opción 2: Descartar tu trabajo local y aceptar íntegramente la versión remota
git checkout --theirs Assets\Project\Scenes\Nivel01.unity
git add Assets\Project\Scenes\Nivel01.unity
git commit -m "resolve: adoptar version remota de Nivel01"
```

Modalidad B: Vía GitHub Desktop

En la modal de conflictos, haz clic en la flecha derecha del archivo y selecciona **Use Modified Version** (tuya) o **Use Existing Version** (remota).

[↑ Volver al Índice de Contenidos](#)

4.6 Tema 6: Cambios Locales en el Editor al Hacer Pull

Si el Editor de Unity guardó automáticamente las escenas al presionar el botón Play y tienes modificaciones sucias que impiden realizar `git pull`:

```
git stash save "guardado-automatico-editor"
git pull --rebase origin main
git stash pop
```

Si al recuperar el stash surgen colisiones en archivos de cache local, descarta los temporales con:

```
git checkout -- Assets\_Project\Scenes\AutoSave.unity
```

[↑ Volver al Índice de Contenidos](#)

4.7 Tema 7: Push Rechazado por Desfase y Rebase Seguro con LFS

Ocurre cuando otro desarrollador subió commits a GitHub mientras tú trabajabas localmente:

```
# 1. Obtener los metadatos y punteros LFS remotos
git fetch origin

# 2. Reorganizar tus commits locales por encima de los remotos
git rebase origin/main

# 3. Subir de forma lineal y limpia
git push origin main
```

El uso estricto de `rebase` mantiene una línea temporal limpia y garantiza que los objetos de Git LFS se descarguen secuencialmente sin merge commits redundantes.

[↑ Volver al Índice de Contenidos](#)

4.8 Tema 8: Conflictos en Binarios y Bloqueo con Git LFS Lock

Los archivos binarios (`.fbx` , `.psd` , `.blend` , `.wav`) no admiten fusión por diferencias de líneas. Para impedir que dos artistas trabajen sobre el mismo archivo a la vez en Windows:

```
POWERSHELL

# 1. Bloquear el archivo en el servidor antes de comenzar a pintar o modelar
git lfs lock Assets\_Project\Art\Personajes\Heroe.psd

# 2. Verificar qué archivos están bloqueados en el repositorio y por qué usuario
git lfs locks

# 3. Trabajar en Photoshop o Blender, guardar, commitear y subir
git add Assets\_Project\Art\Personajes\Heroe.psd
git commit -m "art(heroe): texturizar mascara y detalles de armadura"
git push origin main

# 4. Desbloquear para que el resto del equipo pueda editarlo
git lfs unlock Assets\_Project\Art\Personajes\Heroe.psd
```

[↑ Volver al Índice de Contenidos](#)

4.9 Tema 9: Conflicto de Eliminación de Asset con `.meta` Huérfano

Si un desarrollador elimina un archivo desde el Explorador de Windows pero olvida su `.meta` , Git registrará un conflicto de metadato huérfano:

```
POWERSHELL

# Eliminar metadato huérfano del control de versiones
git rm Assets\_Project\Scripts\OldManager.cs.meta
git commit -m "chore: purgar archivo .meta huerfano tras remocion de script"
```

[↑ Volver al Índice de Contenidos](#)

Parte V: Herramientas Modernas de Productividad Avanzada

5.1 Git Worktrees en Windows: Trabajar en Múltiples Ramas sin Recargar `Library/`

Conmutar de rama en un proyecto de 50 GB obliga al editor a reimportar la carpeta `Library\` durante 20 minutos. Con **Git Worktrees** puedes tener dos ramas abiertas en carpetas independientes del disco en Windows:

```
POWERSHELL

# Crear worktree paralelo en otra carpeta del disco
git worktree add ..\MiJuego-Hotfix hotfix/correccion-audio

# Trabajar en la carpeta secundaria sin tocar el Unity principal
Set-Location ..\MiJuego-Hotfix
git push origin hotfix/correccion-audio

# Al terminar, eliminar el worktree de forma limpia
Set-Location ..\MiJuego
git worktree remove ..\MiJuego-Hotfix
```

Beneficio clave:

Tu instancia principal de Unity no sufre ninguna reimportación de assets ni pierde la caché de compilación de shaders de DirectX/Vulkan.

[↑ Volver al Índice de Contenidos](#)

5.2 Depuración Binaria con Git Bisect y Blame en C#

Cuando una mecánica falla y se desconoce qué commit introdujo la regresión:

```
# Iniciar la búsqueda binaria
git bisect start
git bisect bad           # El commit actual contiene el fallo
git bisect good v1.0.4   # La versión v1.0.4 funcionaba correctamente

# Git cambiará automáticamente de commit intermedio.
# Prueba en Unity y clasifica:
git bisect good # o: git bisect bad

# Al culminar, Git te señalará el commit exacto causante del bug.
git bisect reset
```

[↑ Volver al Índice de Contenidos](#)

5.3 GitHub Codespaces desde PowerShell

Para editar scripts C#, shaders HLSL o archivos de configuración sin necesidad de encender la estación de trabajo completa:

```
gh codespace create --repo mi-organizacion/MiJuegoWindows --branch main
```

Permite revisar PRs, editar código C# con intellisense y compilar librerías en la nube con VS Code en el navegador.

[↑ Volver al Índice de Contenidos](#)

5.4 GitHub Copilot CLI en PowerShell

```
# Consultar comandos complejos de Git para Unity
gh copilot suggest "como revertir un commit de una escena de unity sin tocar scripts"
gh copilot explain "git lfs push --all origin main"
```

[↑ Volver al Índice de Contenidos](#)

5.5 Git Hooks y Validación Pre-commit de Archivos `.meta` en Windows

Crea el archivo `.git\hooks\pre-commit` para evitar que ningún miembro del equipo suba un asset sin su archivo `.meta` :

```
#!/bin/sh
# Pre-commit hook para Windows: Validar integridad de archivos .meta
MISSING=0
for f in $(git diff --cached --name-only --diff-filter=A | grep '^Assets/'); do
    if [ "${f##*.}" != "meta" ]; then
        if [ ! -f "${f}.meta" ] && ! git diff --cached --name-only | grep -q "^${f}.meta$";
    then
        echo "❌ ERROR: Falta el archivo .meta para: $f"
        MISSING=1
    fi
fi
done

if [ $MISSING -eq 1 ]; then
    echo "🚫 Commit rechazado: Todo asset de Unity debe incluir su archivo .meta correspondiente."
    exit 1
fi
exit 0
```

[↑ Volver al Índice de Contenidos](#)

5.6 Descargas Parciales y Ahorro de Cuota de Git LFS con ``lfs.fetchexclude`` y ``git sparse-checkout``

En repositorios masivos de videojuegos de más de 80 GB en Windows, los programadores de lógica o diseñadores de UI no necesitan descargar texturas en 8K ni cinemáticas 4K de niveles en los que no trabajan.

Filtrado de Descarga con `lfs.fetchexclude` en PowerShell



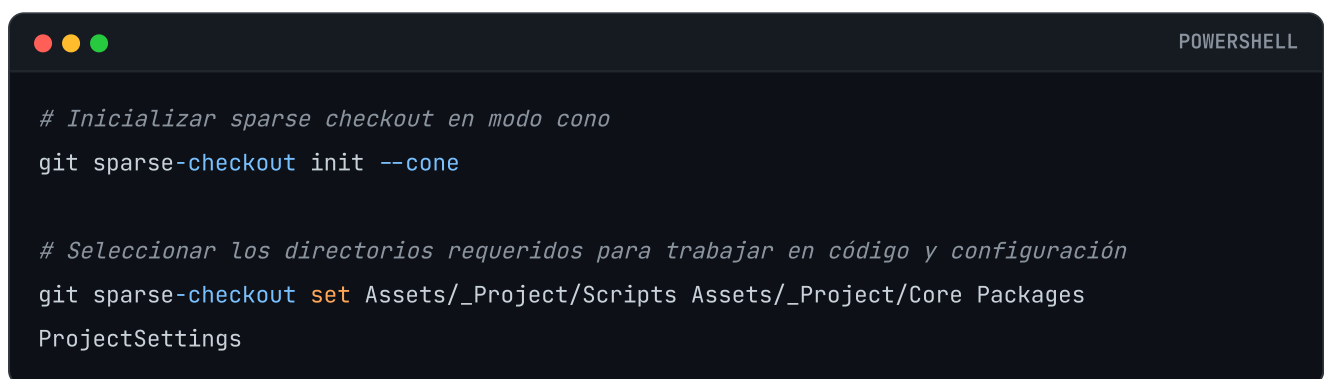
```
# Excluir de la descarga de LFS niveles secundarios y vídeos prerrenderizados
git config lfs.fetchexclude "Assets/_Project/Art/Levels/Nivel04/*,
Assets/_Project/Cinematics/*"

# Descargar únicamente los binarios de interés local
git lfs pull
```

Los assets omitidos permanecerán como punteros ligeros de 130 bytes. Unity compilará el proyecto con normalidad sin consumir almacenamiento masivo innecesario en tu disco SSD.

Sparse-Checkout para Repositorios Gigantes

Descarga únicamente los árboles de carpetas correspondientes a tu rol:



```
# Inicializar sparse checkout en modo cono
git sparse-checkout init --cone

# Seleccionar los directorios requeridos para trabajar en código y configuración
git sparse-checkout set Assets/_Project/Scripts Assets/_Project/Core Packages
ProjectSettings
```

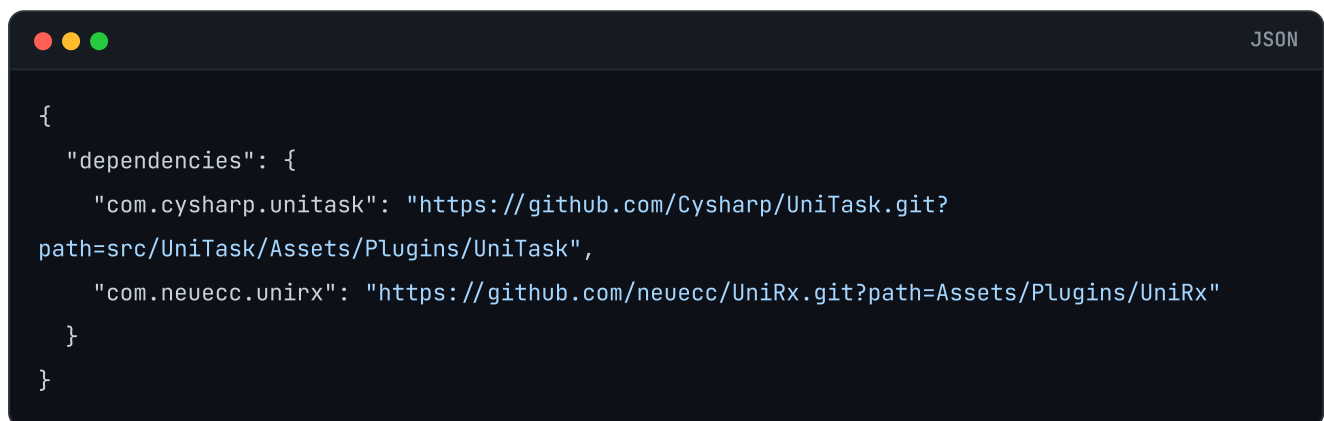
[↑ Volver al Índice de Contenidos](#)

Parte VI: Gestión de Paquetes UPM y Dependencias

6.1 Instalación de Paquetes Mediante URLs de Git en UPM

Unity Package Manager permite consumir paquetes directamente desde repositorios de GitHub.

Añade a `Packages\manifest.json` :

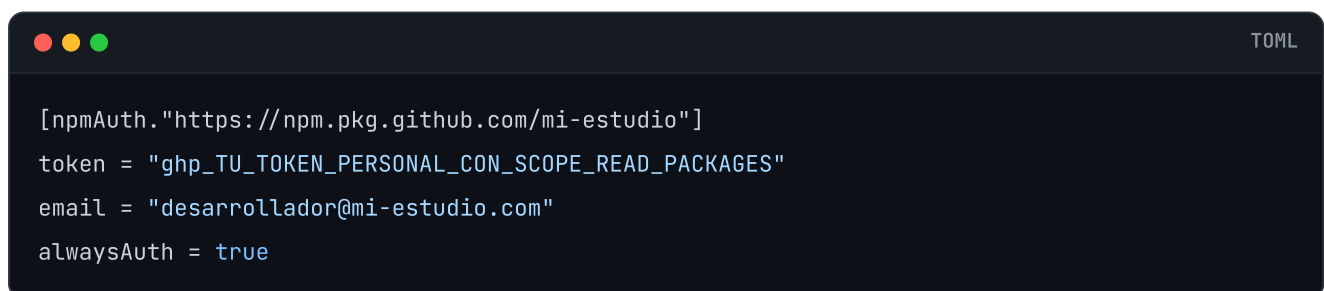
A screenshot of a code editor window with a dark theme. The title bar shows three colored window control buttons (red, yellow, green) on the left and the text 'JSON' on the right. The code is a JSON object with a 'dependencies' key containing two entries: 'com.cysharp.unitask' and 'com.nevecc.unirx', each with a URL pointing to a GitHub repository and a specific path.

```
{
  "dependencies": {
    "com.cysharp.unitask": "https://github.com/Cysharp/UniTask.git?path=src/UniTask/Assets/Plugins/UniTask",
    "com.nevecc.unirx": "https://github.com/nevecc/UniRx.git?path=Assets/Plugins/UniRx"
  }
}
```

[↑ Volver al Índice de Contenidos](#)

6.2 Creación y Publicación de Paquetes UPM en Repositorios Privados con Tokens

Para repositorios privados corporativos en GitHub, configura tu archivo de credenciales de usuario en Windows en `C:\Users\<TuUsuario>\.upmconfig.toml` :

A screenshot of a code editor window with a dark theme. The title bar shows three colored window control buttons (red, yellow, green) on the left and the text 'TOML' on the right. The code is a TOML file containing configuration for a private GitHub repository, including a token, email, and authentication settings.

```
[npmAuth."https://npm.pkg.github.com/mi-estudio"]
token = "ghp_TU_TOKEN_PERSONAL_CON_SCOPE_READ_PACKAGES"
email = "desarrollador@mi-estudio.com"
alwaysAuth = true
```

[↑ Volver al Índice de Contenidos](#)

6.3 Addressables Asset System frente a `Resources/`: Versionado y Despliegue en CDN

El uso de la carpeta `Resources\` es desaconsejado en proyectos comerciales: todos los assets se compilan indivisiblemente en el ejecutable final de Windows, inflando el tamaño del instalador e impidiendo actualizaciones remotas (*live updates*).

La Solución: Addressables (`com.unity.addressables`)

Addressables permite referenciar assets asíncronamente mediante `AssetReference`, compilando AssetBundles para subirlos a AWS S3, Cloudflare R2 o Azure Blob Storage.

Qué se versiona en Git y qué se ignora:

- **En Git:** Versiona `Assets\AddressableAssetSettings\` (definición de grupos, esquemas de compresión y URLs remotas de descarga).
- **Fuera de Git:** Ignora `ServerData\` (la carpeta local donde Unity empaqueta los archivos binarios compilados).

● ● ●

PLAINTEXT

```
# Exclusión de compilación local de Addressables en .gitignore de Windows
[Ss]erver[Dd]ata/
```

[↑ Volver al Índice de Contenidos](#)

Parte VII: Automatización CI/CD con GitHub Actions y GameCI

7.1 Arquitectura de GameCI para StandaloneWindows64

GameCI utiliza imágenes de Docker optimizadas con el Editor de Unity preinstalado para ejecutar pruebas automatizadas y compilar builds de Windows (`.exe`) desatendidas.

[↑ Volver al Índice de Contenidos](#)

7.2 Activación de Licencias de Unity en GitHub Actions

En **Settings** → **Secrets and variables** → **Actions**, define los siguientes secretos:

- `UNITY_EMAIL` : Correo de la cuenta de Unity.
- `UNITY_PASSWORD` : Contraseña de Unity.
- `UNITY_LICENSE` : Contenido en texto del archivo de licencia `.ulf` .

[↑ Volver al Índice de Contenidos](#)

7.3 Pipeline Automatizado: Tests y Compilación StandaloneWindows64 (.exe)

Crea `.github\workflows\build-windows.yml` :

```

name: Unity Windows Build

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    name: 📝 Pruebas Unitarias EditMode y PlayMode
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: true

      - uses: actions/cache@v3
        with:
          path: Library
          key: Library-test-${{ hashFiles('Assets/**', 'Packages/**', 'ProjectSettings/**') }}

      - uses: game-ci/unity-test-runner@v4
        env:
          UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
          UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
          UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
        with:
          githubToken: ${ secrets.GITHUB_TOKEN }

  build:
    name: 🖥️ Compilar StandaloneWindows64 (.exe)
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          lfs: true

      - uses: actions/cache@v3

```

```

with:
  path: Library
  key: Library-Windows-${{ hashFiles('Assets/**', 'Packages/**',
'ProjectSettings/**') }}

- uses: game-ci/unity-builder@v4
  env:
    UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
    UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
    UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
  with:
    targetPlatform: StandaloneWindows64
    buildName: MiJuegoWindows

- uses: actions/upload-artifact@v4
  with:
    name: Build-Windows-x64
    path: build/StandaloneWindows64

```

[↑ Volver al Índice de Contenidos](#)

7.4 Subida Automática de Instaladores a GitHub Releases

El pipeline genera un archivo comprimido descargable con el ejecutable `.exe` y los datos asociados `_Data` de Unity, listos para su distribución a QA o tiendas de videojuegos.

[↑ Volver al Índice de Contenidos](#)

7.5 Pruebas Automatizadas con Code Coverage y Reportes en GitHub Actions

En entornos de producción sobre Windows, todo Pull Request debe verificar la cobertura de pruebas de código mediante `com.unity.test-framework.code-coverage`:

```
- name: Ejecutar Tests con Cobertura de Código
  uses: game-ci/unity-test-runner@v4
  env:
    UNITY_EMAIL: ${ secrets.UNITY_EMAIL }
    UNITY_PASSWORD: ${ secrets.UNITY_PASSWORD }
    UNITY_LICENSE: ${ secrets.UNITY_LICENSE }
  with:
    githubToken: ${ secrets.GITHUB_TOKEN }
    customParameters: -enableCodeCoverage -coverageResultsPath ./coverage-results -
coverageOptions generateAdditionalMetrics;generateHtmlReport;generateBadgeReport

- name: Publicar Reporte de Cobertura
  uses: actions/upload-artifact@v4
  with:
    name: Code-Coverage-Report
    path: ./coverage-results
```

[↑ Volver al Índice de Contenidos](#)

Parte VIII: Seguridad y Políticas de Repositorio en Equipos de Videojuegos

8.1 Branch Protection Rules y Rulesets en Windows

Configura reglas de protección sobre la rama `main` en GitHub:

1. **Require a pull request before merging:** Mínimo 1 aprobación obligatoria de un revisor técnico.
2. **Require status checks to pass:** Exigir que el job `✓ Pruebas Unitarias EditMode y PlayMode` finalice exitosamente.
3. **Block force pushes:** Prohíbe estrictamente `git push --force`.

[↑ Volver al Índice de Contenidos](#)

8.2 Gestión de Secretos para APIs de Juegos (Steam, Photon, Azure)

Guarda las credenciales en variables de entorno o archivos `.env` ignorados por Git. Nunca commitees claves de Steamworks o Azure PlayFab en scripts C#.

[↑ Volver al Índice de Contenidos](#)

8.3 Gobernanza con `CODEOWNERS` para Artistas y Programadores

Crea `.github\CODEOWNERS` para asignar responsabilidades automáticas de revisión:

```
Assets/Scripts/      @lead-programmer
Assets/Art/           @lead-artist
Assets/Scenes/        @level-design-lead
ProjectSettings/      @tech-director
```

[↑ Volver al Índice de Contenidos](#)

Parte IX: Distribución y Despliegue con GitHub Releases

9.1 Creación de Releases con Tags Semánticos desde PowerShell

```
git tag -a v1.0.0 -m "release: version inicial 1.0.0 para Windows"
git push origin v1.0.0
```

[↑ Volver al Índice de Contenidos](#)

9.2 Empaquetado y Publicación de Archivos ZIP e Instaladores de Windows

```
# Crear paquete ZIP en PowerShell
Compress-Archive -Path "build\StandaloneWindows64\*" -DestinationPath "MiJuego-v1.0.0-
Windows.zip"

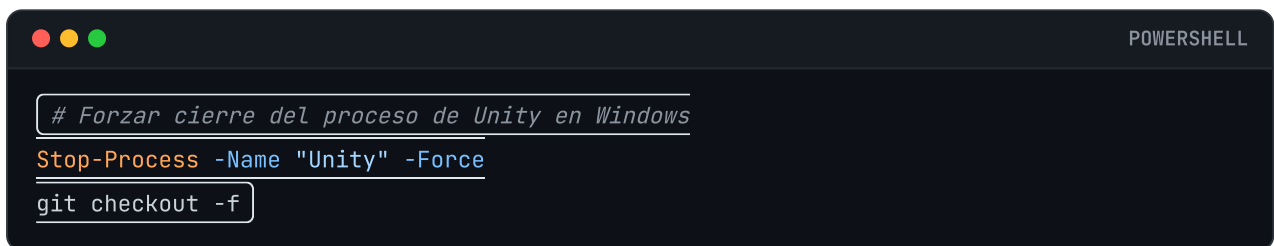
# Publicar el release con GitHub CLI
gh release create v1.0.0 "MiJuego-v1.0.0-Windows.zip" `
  --title "Mi Videojuego v1.0.0 (Windows 64-bit)" `
  --notes "Instalador oficial para Windows 10 y Windows 11."
```

[↑ Volver al Índice de Contenidos](#)

Parte X: Catálogo Maestro de Incidentes Críticos de Unity en Windows

10.1 Incidente 1: Bloqueo de Archivos por el Proceso de Unity (`unlink failed` / `Permission Denied`)

- **Síntoma:** Al ejecutar `git checkout` o `git merge`, la consola muestra: `error: unable to unlink old 'Assets/...': Permission denied` o `file locked by another process`.
- **Causa:** El Editor de Unity o el demonio AssetDatabase mantiene descriptores de archivo abiertos en el sistema NTFS de Windows.
- **Solución:**



```
POWERSHELL

# Forzar cierre del proceso de Unity en Windows
Stop-Process -Name "Unity" -Force
git checkout -f
```

[↑ Volver al Índice de Contenidos](#)

10.2 Incidente 2: Longitud de Ruta Máxima de Windows Excedida (`Filename too long`)

- **Síntoma:** Git arroja el error: `Filename too long` al clonar o extraer paquetes UPM profundamente anidados.
- **Causa:** Límite tradicional MAX_PATH de 260 caracteres en la API de Windows.
- **Solución:**



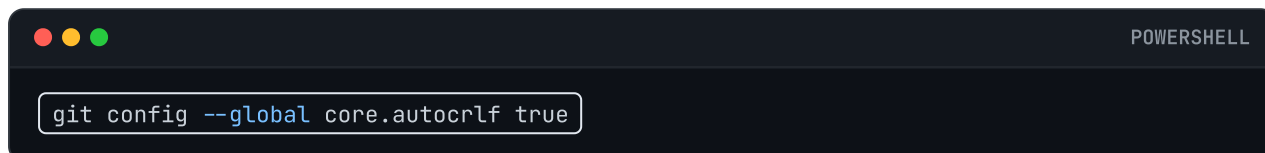
```
POWERSHELL

git config --system core.longpaths true
```

[↑ Volver al Índice de Contenidos](#)

10.3 Incidente 3: Corrupción de Finales de Línea CRLF vs LF en Scripts C# y YAML

- **Síntoma:** Git detecta modificaciones masivas en todos los archivos del proyecto aunque no se haya cambiado ninguna línea de código.
- **Causa:** Windows inserta retorno de carro `\r\n` (CRLF) mientras Unity prefiere saltos de línea UNIX `\n` (LF) para diffs legibles.
- **Solución:**



```
git config --global core.autocrlf true
```

Asegúrate de que `.gitattributes` contenga las reglas `eoL=lf` definidas en la sección 1.6.

[↑ Volver al Índice de Contenidos](#)

10.4 Incidente 4: "Missing Script" Masivo por Desincronización de GUIDs

- **Síntoma:** Múltiples GameObjects en escenas y prefabs muestran en el inspector: `The associated script can not be loaded`.
- **Causa:** Un desarrollador eliminó y recreó el archivo `.meta` de un script C#, asignando un GUID nuevo aleatorio.
- **Solución:**
 1. Busca el GUID previo en el historial de Git:



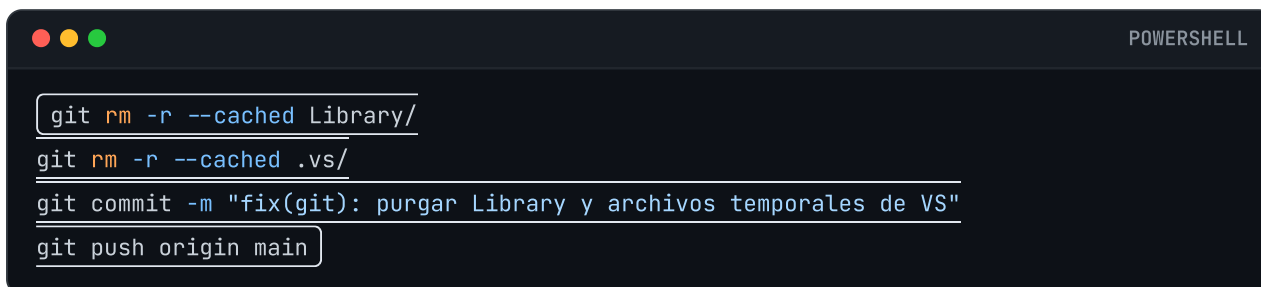
```
git log -p -S "guid:" Assets\_Project\Scripts\Heroe.cs.meta
```

2. Restaura el GUID original en el archivo `.meta` y reinicia Unity con **Assets → Reimport All**.

[↑ Volver al Índice de Contenidos](#)

10.5 Incidente 5: Subida Accidental de `Library/` o Archivos Temporales de Visual Studio

- **Síntoma:** El repositorio pesa varios gigabytes de más y las sincronizaciones con GitHub son extraordinariamente lentas.
- **Solución:**



```
git rm -r --cached Library/
git rm -r --cached .vs/
git commit -m "fix(git): purgar Library y archivos temporales de VS"
git push origin main
```

[↑ Volver al Índice de Contenidos](#)

10.6 Incidente 6: Push Rechazado por Archivo > 100 MB Atrapado en Commits Locales

- **Síntoma:** `remote: error: GH001: Large files detected. File exceeds GitHub's file size limit of 100.00 MB .`
- **Solución con** `git-filter-repo` :



```
pip install git-filter-repo
git lfs migrate import --include="*.fbx,*.psd,*.blend,*.wav" --everything
git push origin --force --all
```

[↑ Volver al Índice de Contenidos](#)

10.7 Incidente 7: Límite de Ancho de Banda de Git LFS Superado

- **Síntoma:** `Git LFS: Repository or organization has exceeded its bandwidth or storage quota .`
- **Solución:**

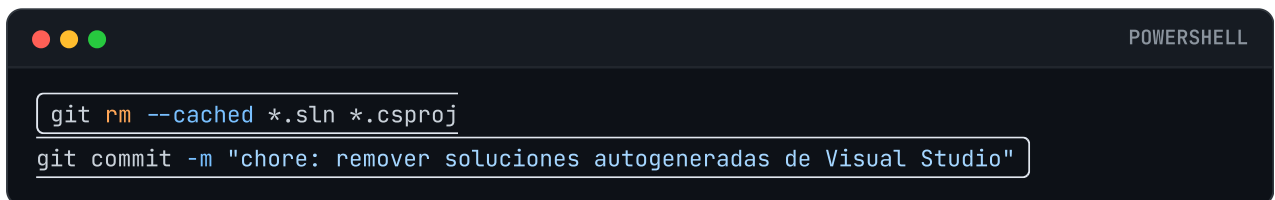
A screenshot of a Windows PowerShell terminal window. The title bar shows three colored window control buttons (red, yellow, green) on the left and the word "POWERSHELL" on the right. The terminal has a dark background. Two lines of text are visible, each enclosed in a light blue rounded rectangle: "git lfs prune --dry-run" and "git lfs prune".

```
git lfs prune --dry-run
git lfs prune
```

[↑ Volver al Índice de Contenidos](#)

10.8 Incidente 8: Conflictos de Visual Studio con `.sln` y `.csproj` en Git

- **Síntoma:** Constantes conflictos en archivos de solución de Visual Studio al hacer pull.
- **Causa:** Los archivos `.sln` y `.csproj` se autogeneran por Unity y nunca deben versionarse.
- **Solución:**

A screenshot of a Windows PowerShell terminal window. The title bar shows three colored window control buttons (red, yellow, green) on the left and the word "POWERSHELL" on the right. The terminal has a dark background. Two lines of text are visible, each enclosed in a light blue rounded rectangle: "git rm --cached *.sln *.csproj" and "git commit -m 'chore: remover soluciones autogeneradas de Visual Studio'".

```
git rm --cached *.sln *.csproj
git commit -m "chore: remover soluciones autogeneradas de Visual Studio"
```

[↑ Volver al Índice de Contenidos](#)

10.9 Incidente 9: Fuga de Claves de Steamworks o Azure en ScriptableObject

- **Síntoma:** Se subió accidentalmente una API key o secreto de autenticación a GitHub.
- **Solución:** Revoca el token inmediatamente en el portal del desarrollador de Steam o Azure y purga el archivo del historial con `git filter-repo`.

[↑ Volver al Índice de Contenidos](#)

10.10 Incidente 10: Regeneración Segura y Limpia de la Caché Local en Windows

- **Síntoma:** Errores inexplicables de compilación en C#, referencias rotas o fallos de renderizado que solo le ocurren a una persona en el equipo.
- **Solución (El Reset Nuclear Seguro):**

POWERSHELL

```
# 1. Forzar cierre de Unity
Stop-Process -Name "Unity" -Force -ErrorAction SilentlyContinue

# 2. Eliminar carpetas temporales locales en Windows
Remove-Item -Recurse -Force Library, Temp, Obj, Logs, UserSettings

# 3. Abre el proyecto nuevamente desde Unity Hub.
# Unity reconstruirá la base de datos de Library limpia basándose en Assets y Packages.
```

[↑ Volver al Índice de Contenidos](#)